Chapter 5a
Input/Output

- Principles of I/O hardware
- Principles of I/O software
- I/O software layers
- Disks

OS and I/O

- One of the main functions of OS is to control all the I/O devices
  - Issue commands to the devices
  - Catch interrupts
  - Handle errors
  - Provide an interface between the devices and the rest of OS
    - Make device independent

I/O Hardware

- Hardware views
  - Chips, circuits, power supplies, motors, wires, etc
    - This is inside of a device (electrical engineers concerned)
  - Programming I/O devices is what an OS designer interested in
    - The commands the hardware accepts
    - The functions it carries out
    - The errors reported back…

I/O Devices

- Incredible variety of I/O devices
  - Storage Devices (Disk, tape)
  - Transmission Devices (Network Card, Modem)
  - Human Interface Devices (Screen, Keyboard, mouse)
  - Specialized Devices (Joystick)
- Devices Classification
  - Block or character-stream
  - Sequential or random-access
  - Synchronous or asynchronous devices
  - Sharable or dedicated
  - read-write, read only, or write only
  - Speed of operation

I/O Devices

- Block Devices
  - Stores information in fixed-size blocks, each one with its own address
  - 512B~32KB per block
  - It's possible to read or write each block independently of all the other ones
  - E.g. Disks
- Character device
  - Delivers or accepts a stream of characters
  - Not addressable and no seek operation
  - E.g. Printers, network interfaces, mouse, etc

I/O Devices

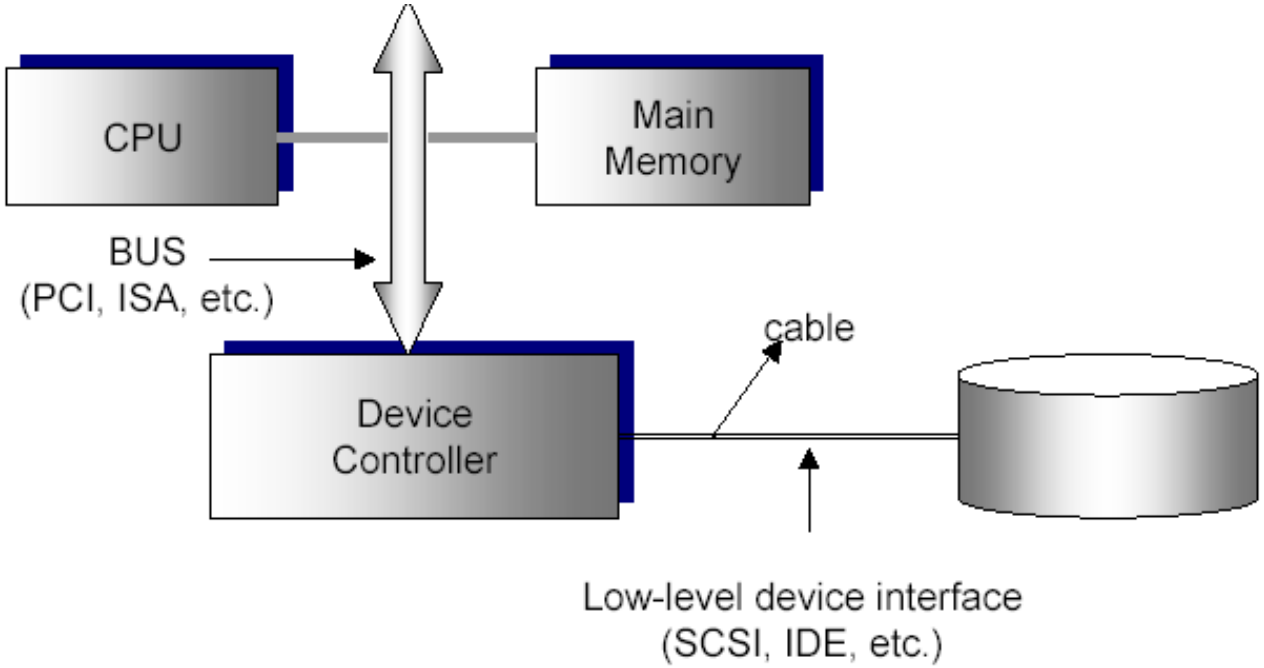
Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner	400 KB/sec
Digital camcorder	3.5 MB/sec
802.11g Wireless	6.75 MB/sec
52x CD-ROM	7.8 MB/sec
Fast Ethernet	12.5 MB/sec
Compact flash card	40 MB/sec
FireWire (IEEE 1394)	50 MB/sec
USB 2.0	60 MB/sec
SONET OC-12 network	78 MB/sec
SCSI Ultra 2 disk	80 MB/sec
Gigabit Ethernet	125 MB/sec
SATA disk drive	300 MB/sec
Ultrium tape	320 MB/sec
PCI bus	528 MB/sec

Some typical device, network, and bus data rates

Device Controllers

- I/O devices have components:
  - mechanical component (the device itself)
  - electronic component
- The electronic component is the **device controller** or **adapter**.
  - On PCs, it often takes the form of a chip on the parentboard or a printed circuit card that can be inserted into an expansion **slot**.
  - May be able to handle multiple devices

Device Controllers



- Controller's tasks (e.g. disk)
  - convert serial bit stream to block of bytes
  - perform error correction as necessary
  - make available to main memory

Device Controllers

- Each controller have a few registers that are used for communicating with the CPU.
- The operating system can command the device by writing into these registers and learn the device's state by reading from these registers.



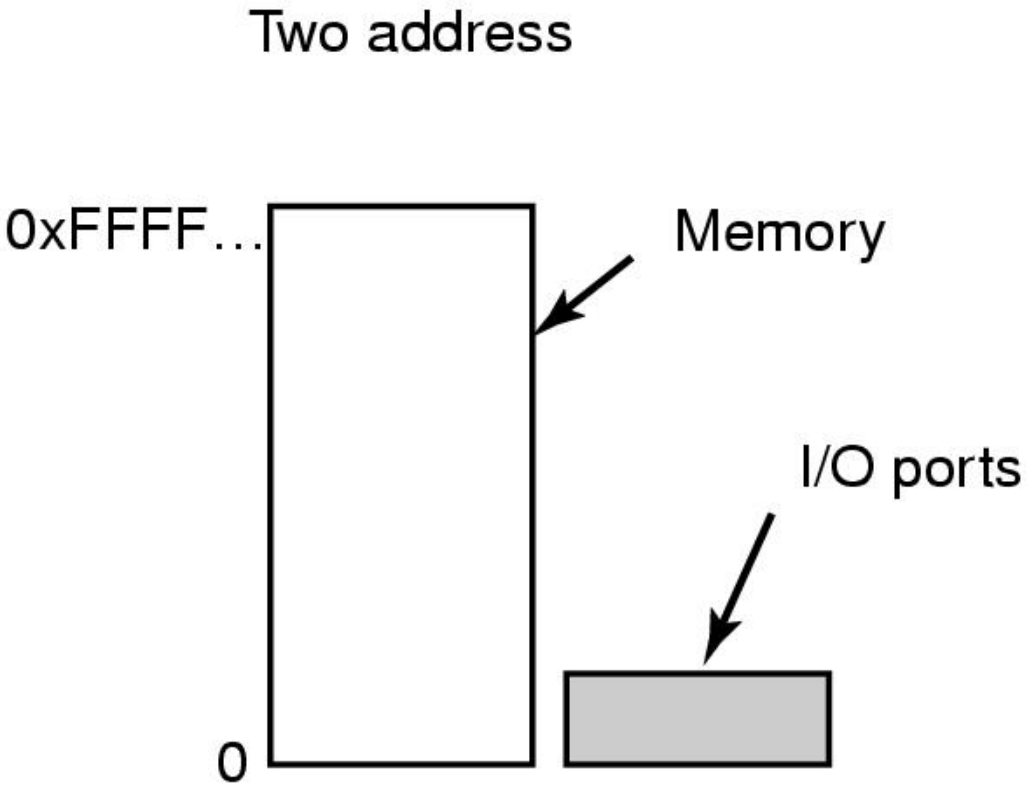
Device Controllers

- Problems:
  - How to addresses these control registers and data buffers
  - How to transfer data between controller and memory
- Methods:
  - Separate I/O and memory space
  - Memory-mapped I/O
  - Hybrid Scheme

Separate I/O and memory space

- Each control register is assigned a special address called **I/O port** number, an 8- or 16-bit integer.
- Special I/O instructions are used to read/write from/into these control registers
  - IN REG, PORT
    - Read a byte or word from control register addresses as PORT into the CPU register REG.
  - OUT REG, PORT
    - Write the content of CPU register REG into the control register addresses as PORT.
- Separate I/O and memory space
- Most early computers worked this way

Separate I/O and memory space



IN R0, 4
MOV R0, 4
are completely different.

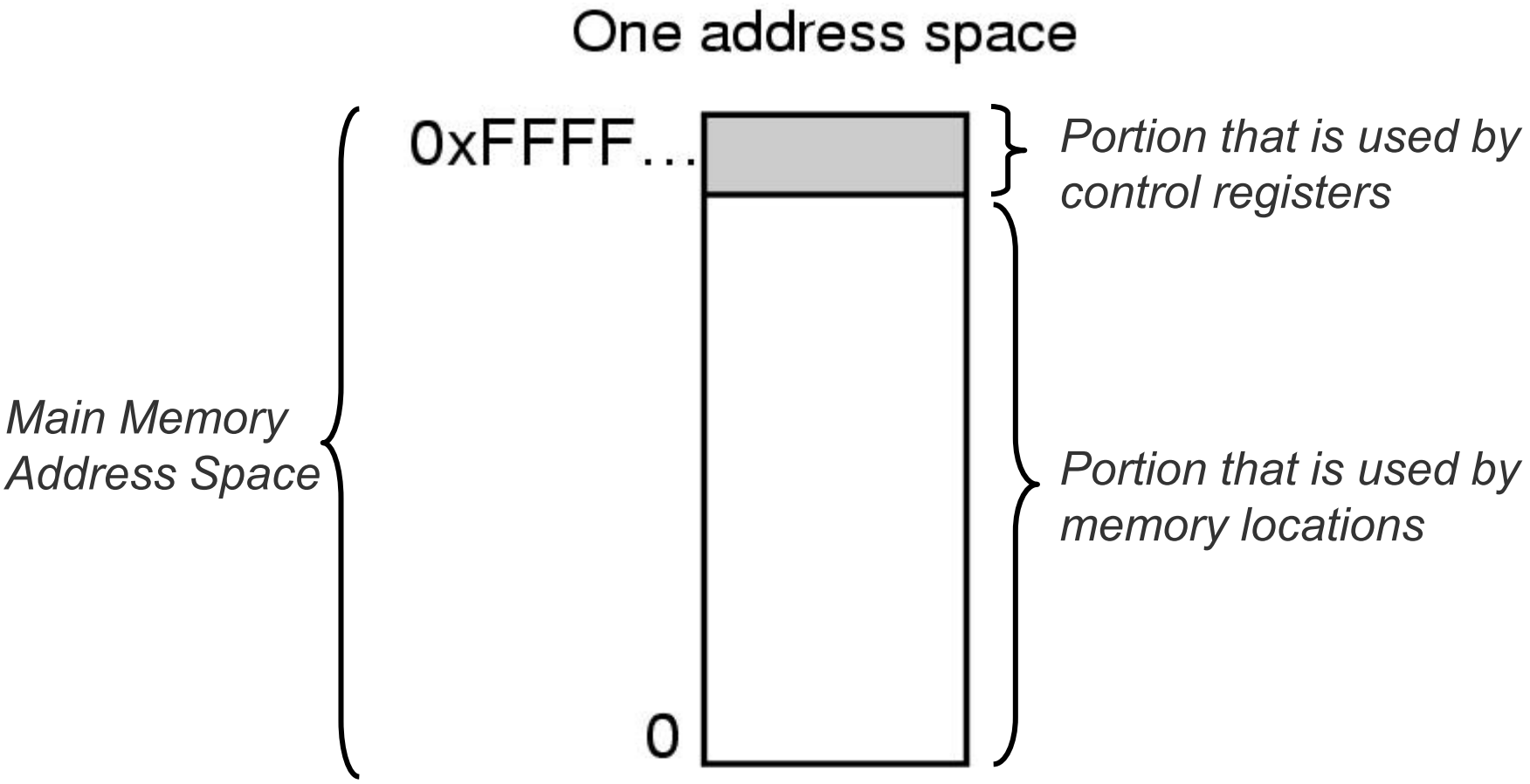
- 4 in the first instruction refers to control register whose address is 4.
- 4 in the second instruction refers to main memory location whose address is 4

- Separate I/O and memory space (e.g. IBM 360)

Memory-mapped I/O

- All the control registers are mapped into the memory space, but separating I/O ports from memory
- Controller Registers are part of regular address space
- Each control register is assigned a unique memory address to which no physical memory is assigned
- Usually the addresses assigned to control registers are on top of the address space
- Usually flags in special registers indicate that the memory access is for memory-mapped I/O

Memory-Mapped I/O

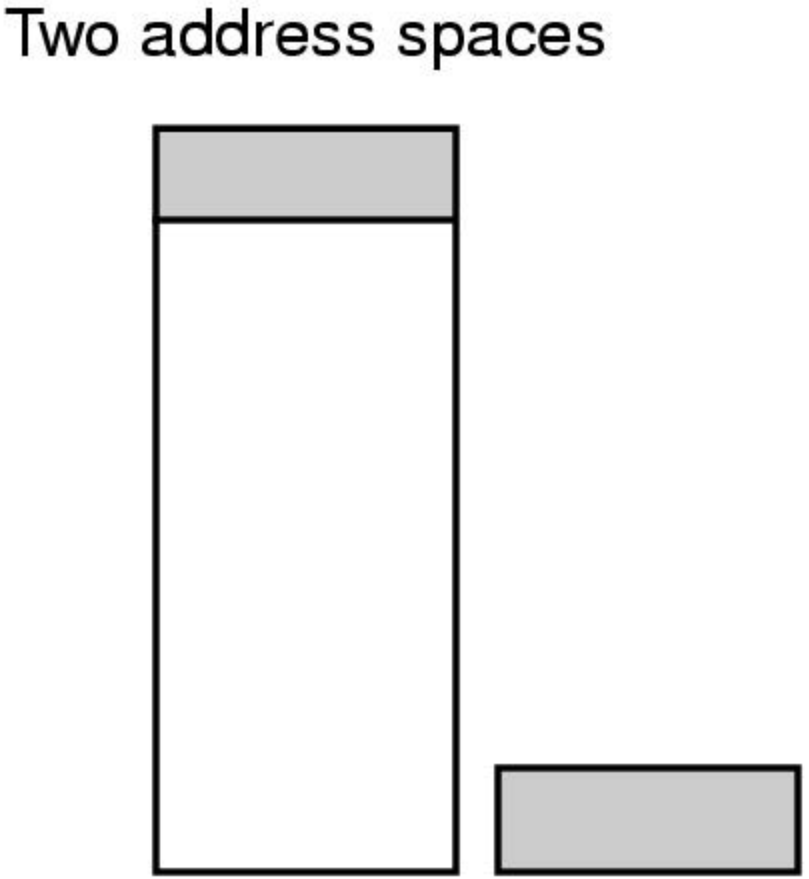


- Memory-mapped I/O (e.g. PDP-11)

Hybrid Scheme

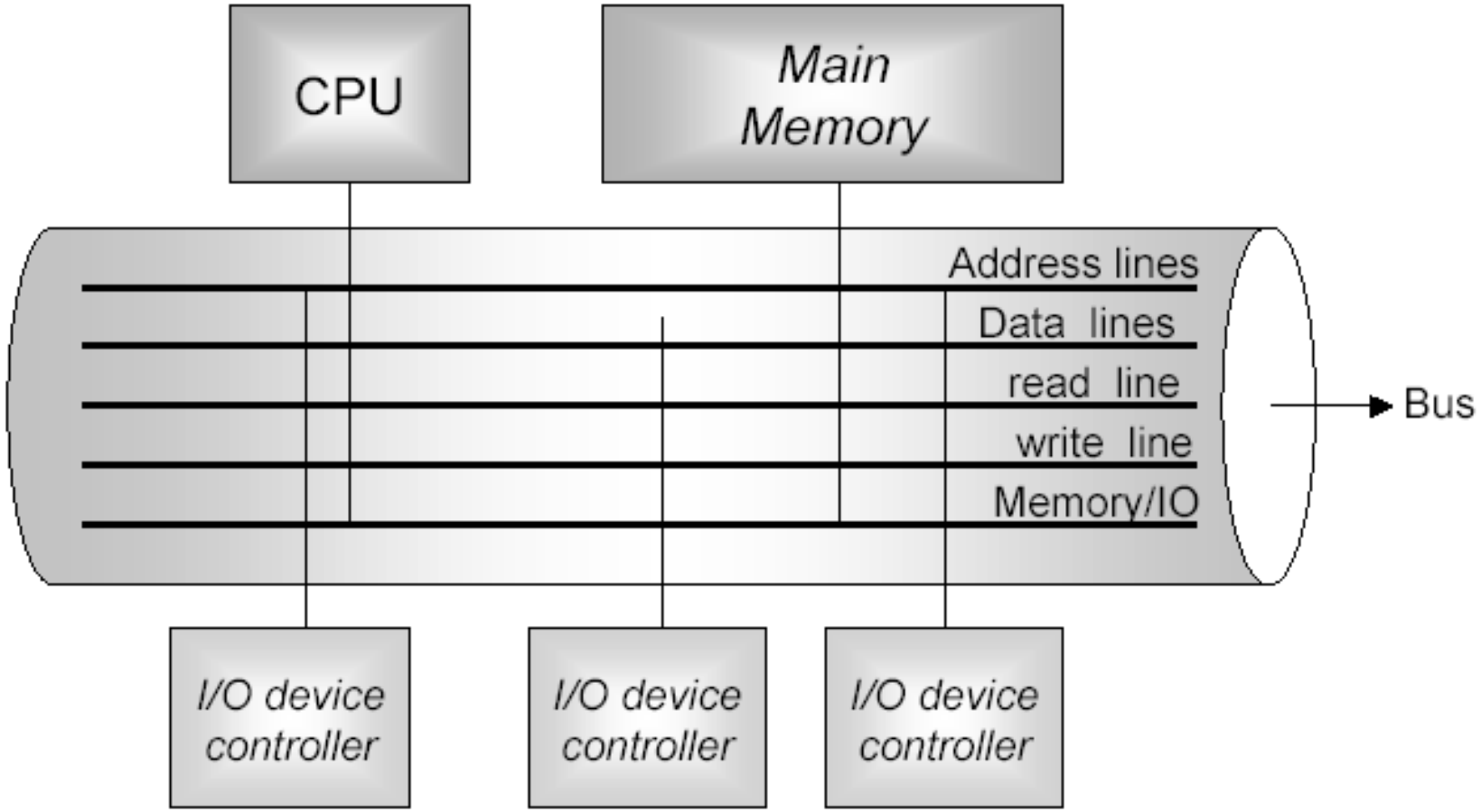
- Data buffers use memory-mapped addresses
- Control registers use addresses from a separate address space
  - Pentium use this scheme
    - 0~64K: I/O port address space
    - 640K~1M: reserved main memory address space part that is used by data buffers

Hybrid Scheme



- Hybrid scheme (e.g. Pentium)

How these schemes work



How these schemes work
- Separate I/O and memory space

- Assume we will do a read operation
- CPU puts the address on the bus
- CPU activates the read line
- CPU asserts a signal line that says whether we will read from memory or I/O device
- If we will read from memory
  - Address lines contain a memory address from the memory address space
  - Main Memory responds.
- If we will read from I/O device
  - Address lines contain an address from I/O port address space.
  - Respective device controller responds

How these schemes work
- Memory-mapped I/O

- Assume we will do a read operation
- CPU puts the address on the bus
- CPU activates the read line
- If we will read from memory
  - Address lines contain an address from part of address space for main memory
  - Main Memory responds.
- If we will read from I/O device
  - Address lines contain an address from part of address space for device controllers
  - Respective device controller responds

Advantages of Memory-mapped I/O

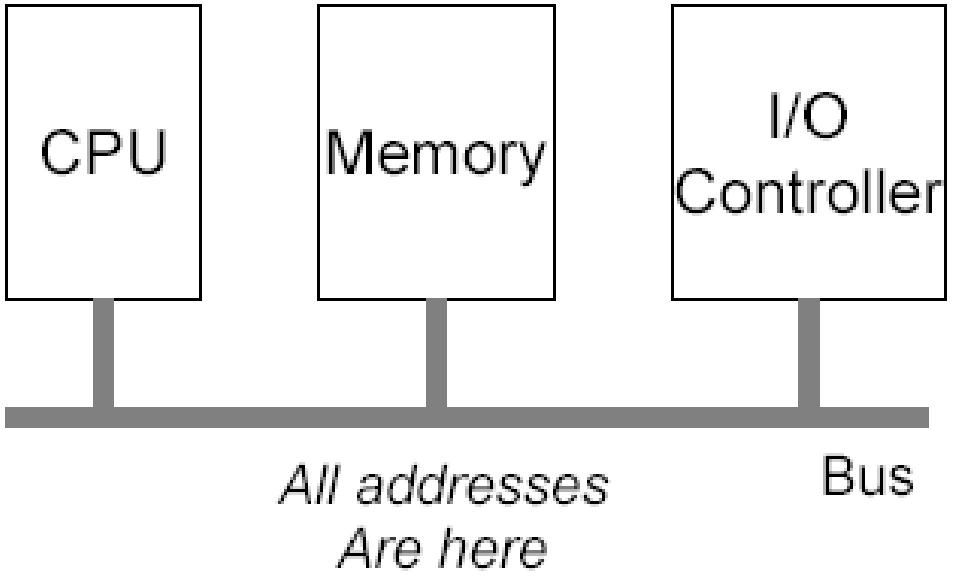
- No special instructions are needed to read and write from/to control registers
- No special protection mechanism is necessary to protect control registers from direct access of users
- Every instruction that references memory can also reference control registers.

Disadvantages of Memory-mapped I/O

- Caching
  - If device controller register values are caches, this can be problematic.
    - We don't want to read cached soft copy, but we want to read the control register content.
    - Solution: Disable caching for some virtual pages that correspond to control registers.
- Because of only one address space, all main memory modules and all I/O device controllers must examine all memory references to see which one to respond to.

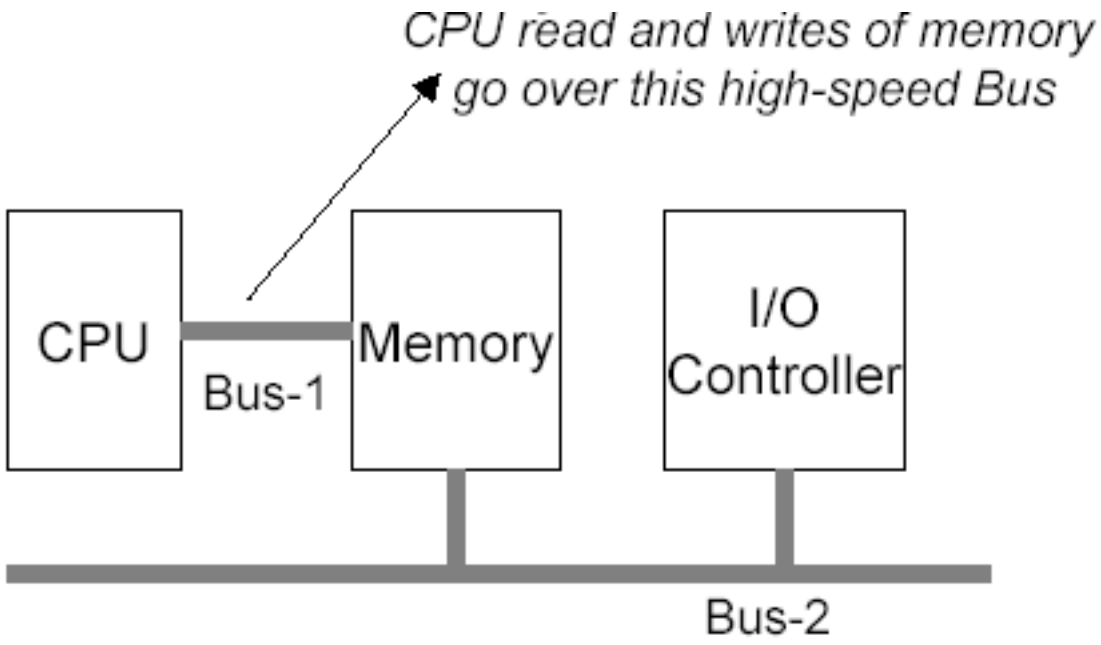
Disadvantages of Memory-mapped I/O

- This can be easy or difficult depending on the architecture of the system
  - No problem in this single-bus architecture



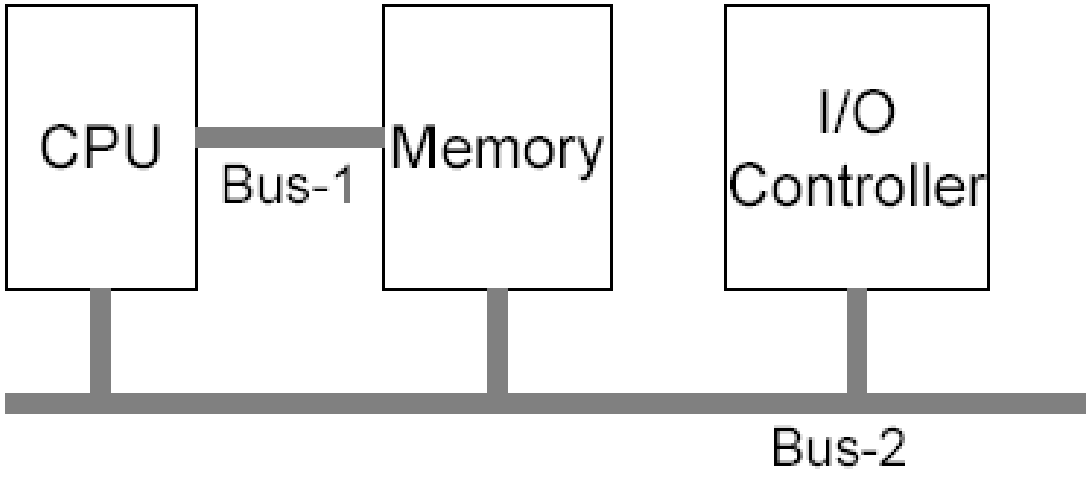
Disadvantages of Memory-mapped I/O

- This can be easy or difficult depending on the architecture of the system
  - In this dual-bus memory architecture, how can I/O controllers see the addresses that are used for control registers, as all memory addresses go by on the memory bus.



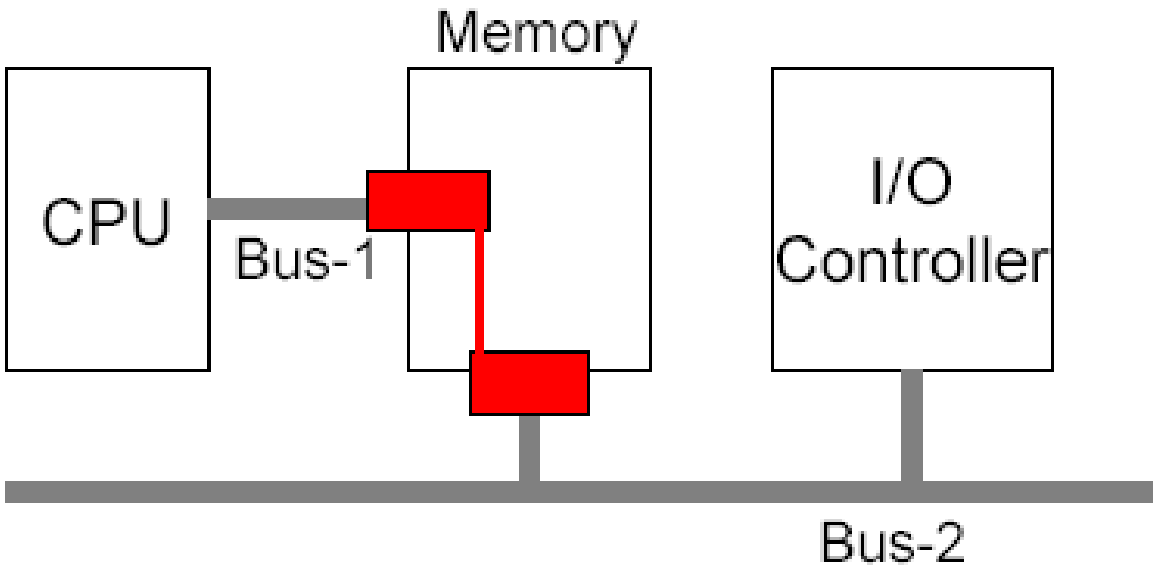
Disadvantages of Memory-mapped I/O

- Solution 1
  - CPU sends an address to main memory over the memory bus. If main memory does not respond (since address range is for I/O), the CPU will put the address on the other buses.



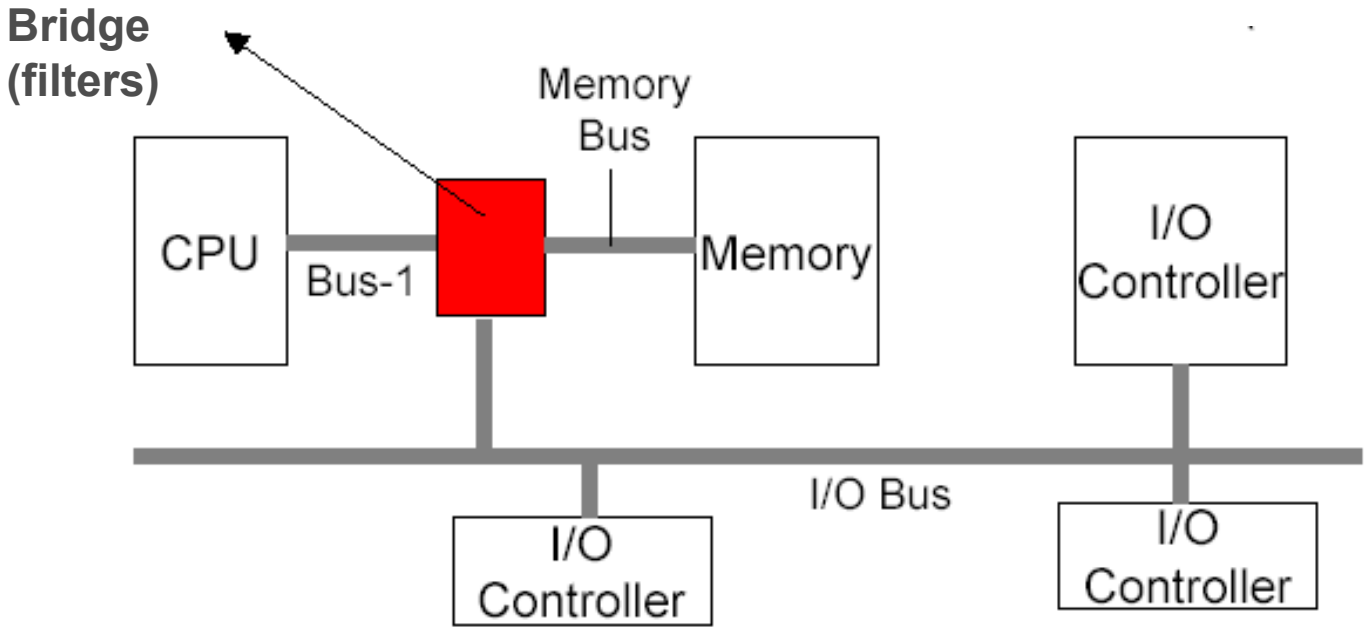
Disadvantages of Memory-mapped I/O

- Solution 2
  - A snooping device on the memory bus can be used to pass all addresses in I/O range to the I/O bus.



Disadvantages of Memory-mapped I/O

- Solution 3
  - A bridge filters all memory addresses. If addresses are in main memory address range, then it passes the addresses to the memory bus; if addresses are in I/O space then it passes the addresses to the I/O bus. (e.g. Pentium)



Principles of I/O Software

- Goals of I/O Software
- I/O Operations
  - Programmed I/O
  - Interrupt-driven I/O
  - I/O using DMA

Goals of I/O Software (1)

- Device independence
  - programs can access any I/O device
  - without specifying device in advance
    - (USB device, hard drive, or CD-ROM)
- Uniform naming
  - name of a file or device a string or an integer
  - not depending on which machine
- Error handling
  - handle as close to the hardware as possible

Goals of I/O Software (2)

- Synchronous vs. asynchronous transfers
  - blocked transfers vs. interrupt-driven
- Buffering
  - data coming off a device cannot be stored in final destination
- Sharable vs. dedicated devices
  - disks are sharable
  - tape drives would not be

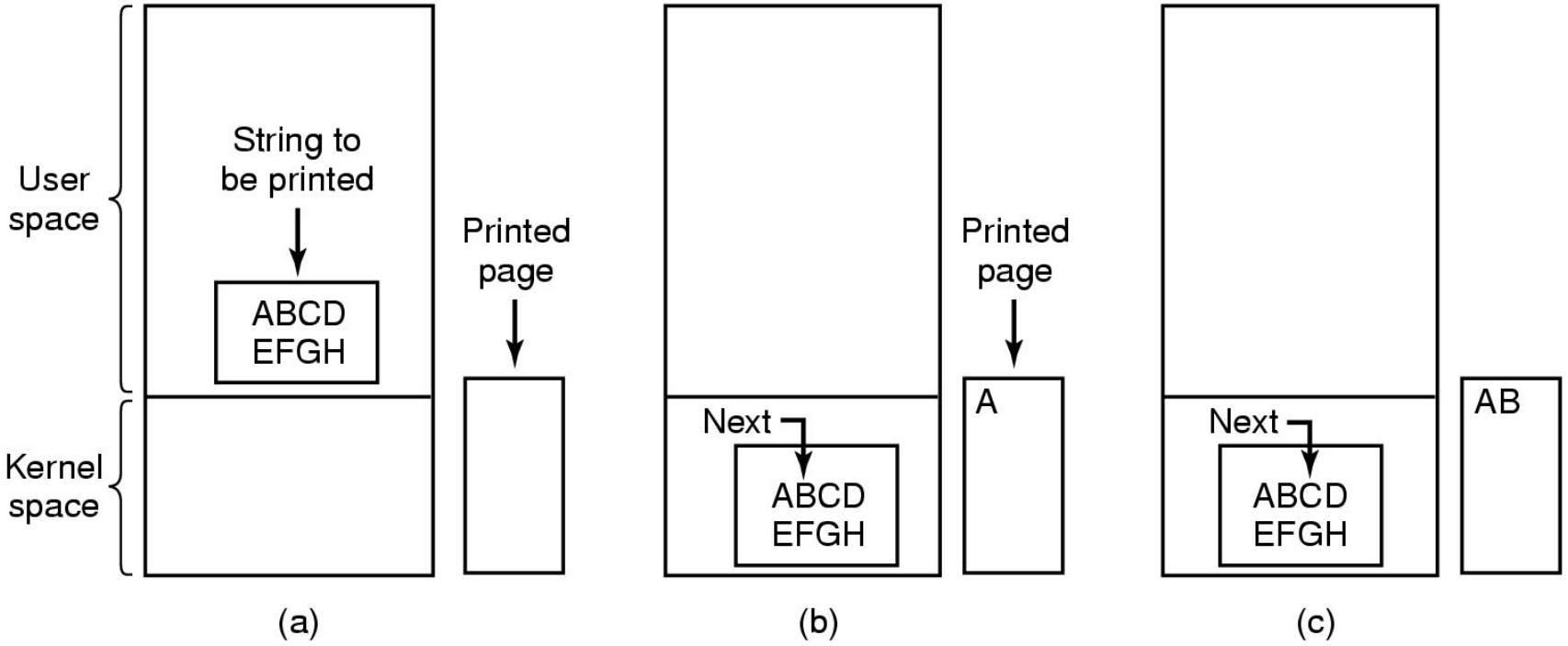
Programmed I/O (1)

- CPU is always busy with I/O until I/O gets completed.
  - Fine for single process systems
    - MS-DOS
    - Embedded systems
  - But not a good approach for multi-programming and time-sharing systems

Programmed I/O (2)

- Assume a program that wants to print a string to a printer.
- Assume string is “ABCDEFGH”, 8 character long string
- Assume printer has 1 byte of data buffer to put the character that needs to be printed.

Programmed I/O (3)



Steps in printing a string

Programmed I/O (4)

```
/*
buffer is user buffer
p is kernel buffer
count is number of bytes to be copied
*/
copy_from_user(buffer, p, count);                  /* p is the kernel bufer */
for (i = 0; i < count; i++) {                      /* loop on every character */
        while (*printer_status_reg != READY) ;     /* loop until ready */
        *printer_data_register = p[i];             /* output one character */
}
return_to_user( );
```

Writing a string to the printer using programmed I/O

Programmed I/O (5)

- Polling: CPU busy with checking the status register of printer to determine its state
  - Ready
  - Busy
  - Error
- Busy-wait cycle to wait for I/O from device, waste of CPU

Interrupt-Driven I/O (1)

- After copying application buffer content into kernel buffer, and sending 1 character to printer,
  - CPU does not wait until printer is READY again.
  - Instead, the scheduler() is called and some other process is run.
    - The current process is put into blocked state.
  - Printer controller generates an hardware interrupt:
    - When the character is written to the printer and the printer becomes READY again.

Interrupt-Driven I/O (2)

```
copy_from_user(buffer, p, count);
enable_interrupts( );
while (*printer_status_reg != READY) ;
*printer_data_register = p[0];
scheduler( );
```

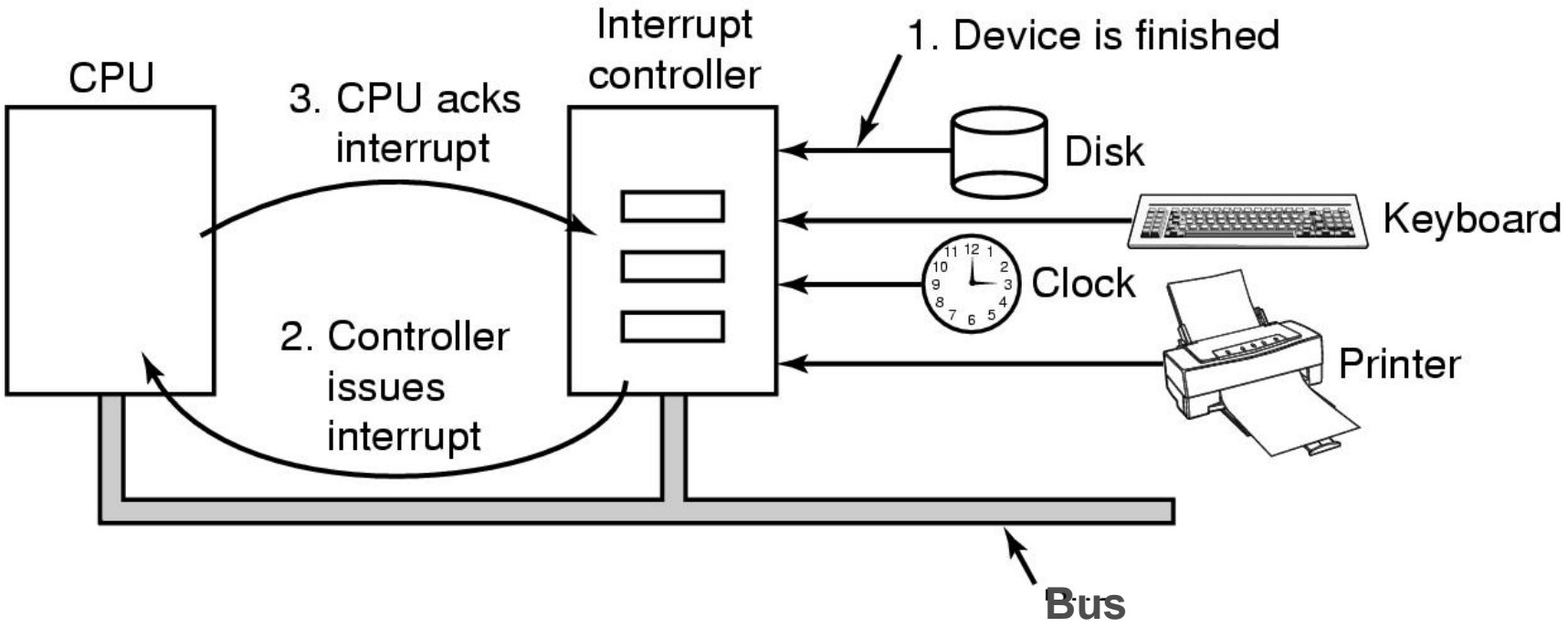
(a)

```
if (count == 0) {
    unblock_user( );
} else {
    *printer_data_register = p[i];
    count = count – 1;
    i = i + 1;
}
acknowledge_interrupt( );
return_from_interrupt( );
```

(b)

- Writing a string to the printer using interrupt-driven I/O
  (a) Code executed when print system call is made
  (b) Interrupt service procedure

Interrupt



How interrupts happens. Connections between devices and interrupt controller actually use interrupt lines on the bus rather than dedicated wires

I/O using DMA (1)

- Disadvantage of Interrupt-Driven I/O is that interrupt occurs on every character.
- DMA controller will feed the characters from kernel buffer to the printer controller.
  - CPU is not bothered during this transfer
- After transfer of whole buffer (string) is completed, then the CPU is interrupted.

I/O using DMA (2)

```
copy_from_user(buffer, p, count);
set_up_DMA_controller( );
scheduler( );
```

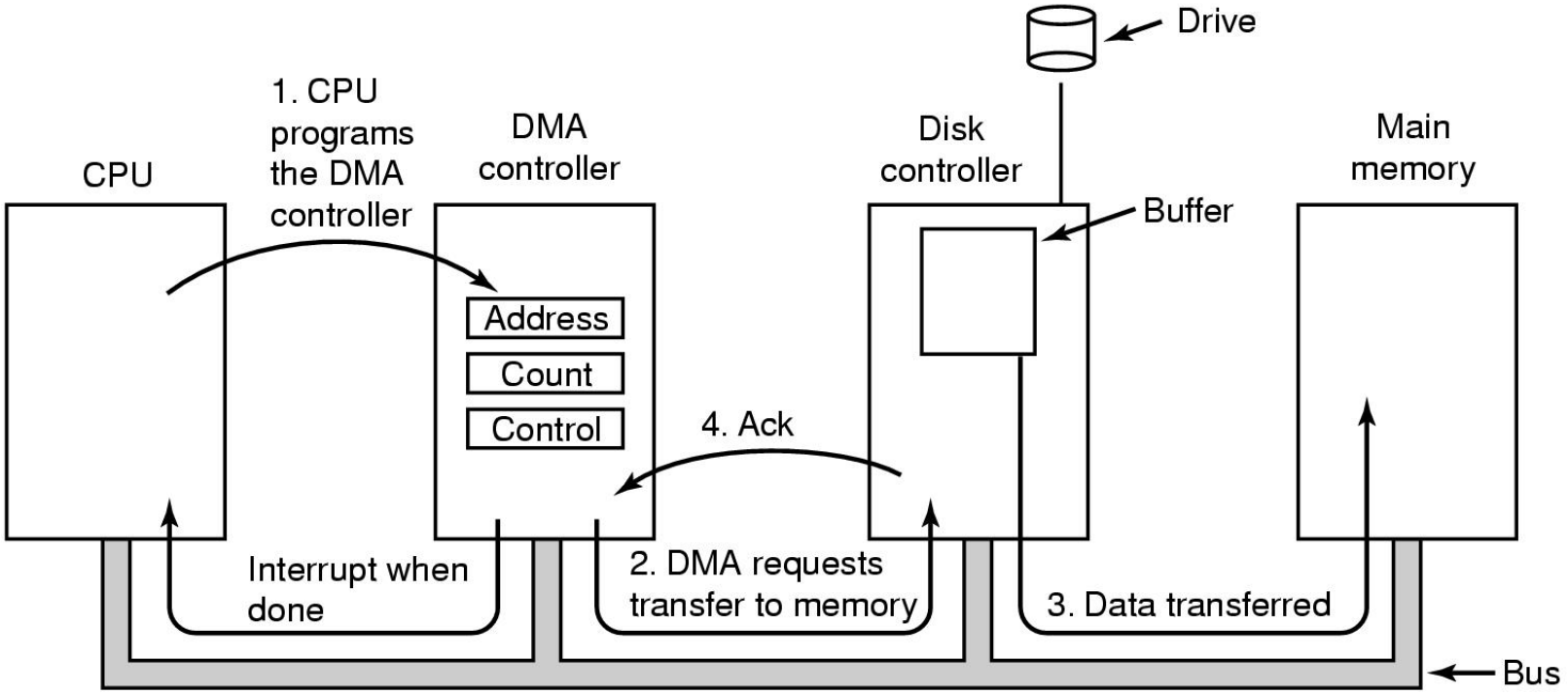
(a)

```
acknowledge_interrupt( );
unblock_user( );
return_from_interrupt( );
```

(b)

- Printing a string using DMA
  - code executed when the print system call is made
  - interrupt service procedure

Direct Memory Access (DMA)



Operation of a DMA transfer